University of the Algarve

Faculty of Science and Technology

Masters in Informatics Engineering

Computer Games

Alive till Nightfall

Development and Implementation of a 3D Tag Game

Alexandre Boavida, a68781

Phillipe Schlup, a73820

Jan-Erik Voigt, a95650

1st of July, 2026

Abstract

“Alive by Nightfall” is a 3D asymmetric game developed in the Unity game engine. It pits a human player against a machine opponent in a series of timed rounds where roles alternate between Hunter and Runner. This report details the game mechanics, technical architecture, level design progression across three distinct arenas, and the user interface implementation. It highlights the use of robust object-oriented programming principles, a centralized state management system, and advanced artificial intelligence pathfinding to create a seamless gameplay experience.

Contents

List of Figures

Introduction

“Alive till Nightfall” is a 3D game developed using the Unity engine. It explores asymmetric multiplayer mechanics by pitting a human player against a machine-controlled opponent, originally inspired by the popular game “Dead by Daylight” the game attempts to mimic the gameplay of a player who runs and one who chases. The core gameplay loop revolves around a digital game of tag, heavily relying on spatial awareness, navigation, and time management. It utilizes custom physics, input handling, and a dynamic level progression system to provide an engaging and structured experience.

Gameplay and Mechanics

The game features two distinct roles: the Hunter and the Runner. The Hunter's objective is to navigate the arena and physically collide with the Runner before the time expires. Conversely, the Runner must utilize the environment to evade capture until the round timer reaches zero.

It is structured in a "Best-of-5" format. Each round lasts a maximum of 30 seconds. To ensure fairness and variability, the roles are swapped at the beginning of every new round. The first entity human or machine to achieve three victories is declared the winner of the match.

Control of the human character is handled via standard PC inputs: `W` to move forward, `S` to move backward, and `A` / `D` to rotate the capsule left and right, respectively.

Technical Architecture

Game Management

The core state of the game is governed by the `GameManager` script, which implements the Singleton pattern to ensure only one instance dictates the game flow. It tracks the current round, human and machine scores, total time played, and the active roles. It is also responsible for resetting character positions, handling round transitions, and toggling the appropriate UI panels.

State Machine and Character Controllers

Movement and interaction are governed by a State Machine pattern. A centralized `GameCharacter` script acts as the physical entity, dynamically delegating logic to either a `HumanPlayerController` or a `MachinePlayerController` via an interface (`IPlayerController`). The human controller relies on Unity's modern Input System and a `Rigidbody` component, utilizing `MovePosition` and `MoveRotation` to ensure solid interactions with walls. When control is passed to the machine, the Rigidbody is set to kinematic, allowing the AI to take over navigation without physical conflicts.

Artificial Intelligence Navigation

The machine opponent utilizes Unity's Navigation Mesh (NavMesh) system for intelligent pathfinding across the varied arenas. The AI behavior adapts strictly to its current role:

- **Hunter Behavior:** The AI aggressively tracks the human player by continuously setting its NavMeshAgent destination to the opponent's coordinates.
- **Runner Behavior:** The AI executes a dynamic hiding algorithm. It uses spatial queries (`Physics.OverlapSphere`) to detect nearby obstacles and calculates candidate hiding spots on the opposite side of these obstacles relative to the Hunter. It then performs checks (`NavMesh.SamplePosition`) to ensure the spot is walkable, and line-of-sight checks (`Physics.Linecast`) to confirm the spot breaks visual contact. The algorithm scores these candidates based on safety distances, ultimately directing the agent to the optimal hiding location.

Camera Dynamics

To maintain focus on the action, it employs a custom `CameraFollow` script. During active gameplay, the camera uses linear interpolation (`Vector3.Lerp`) to smoothly trail behind the active human player, calculating a desired offset based on distance and height parameters. Furthermore, a `SnapToTarget` method is invoked during map transitions, instantly teleporting the camera to prevent jarring visual panning across the scene when the arenas are swapped.

Level Design and Progression

It features three distinct arenas, introducing visual and spatial variety as the match progresses. The `GameManager` dynamically loads the appropriate map based on the current round, utilizing pre-defined `ArenaSetup` configurations to securely teleport the capsules to designated `Transform` spawn points.

- **Rounds 1 and 2:** The match begins in Arena 1. It establishes the baseline mechanics with a specific layout designed for immediate engagement.
- **Rounds 3 and 4:** The game transitions to Arena 2. This environment features more complex collision geometry, requiring the use of properly configured Mesh Colliders and NavMesh surfaces to ensure accurate interaction with uneven terrain and varied obstacles.
- **Round 5 (Tiebreaker):** The final showdown occurs in Arena 3, a specialized map designed for the concluding tension of a tied match.

To ensure stable physics, all imported 3D models were stripped of conflicting transform histories (such as negative scales), and custom invisible physics boundaries were implemented to prevent characters from falling out of bounds.

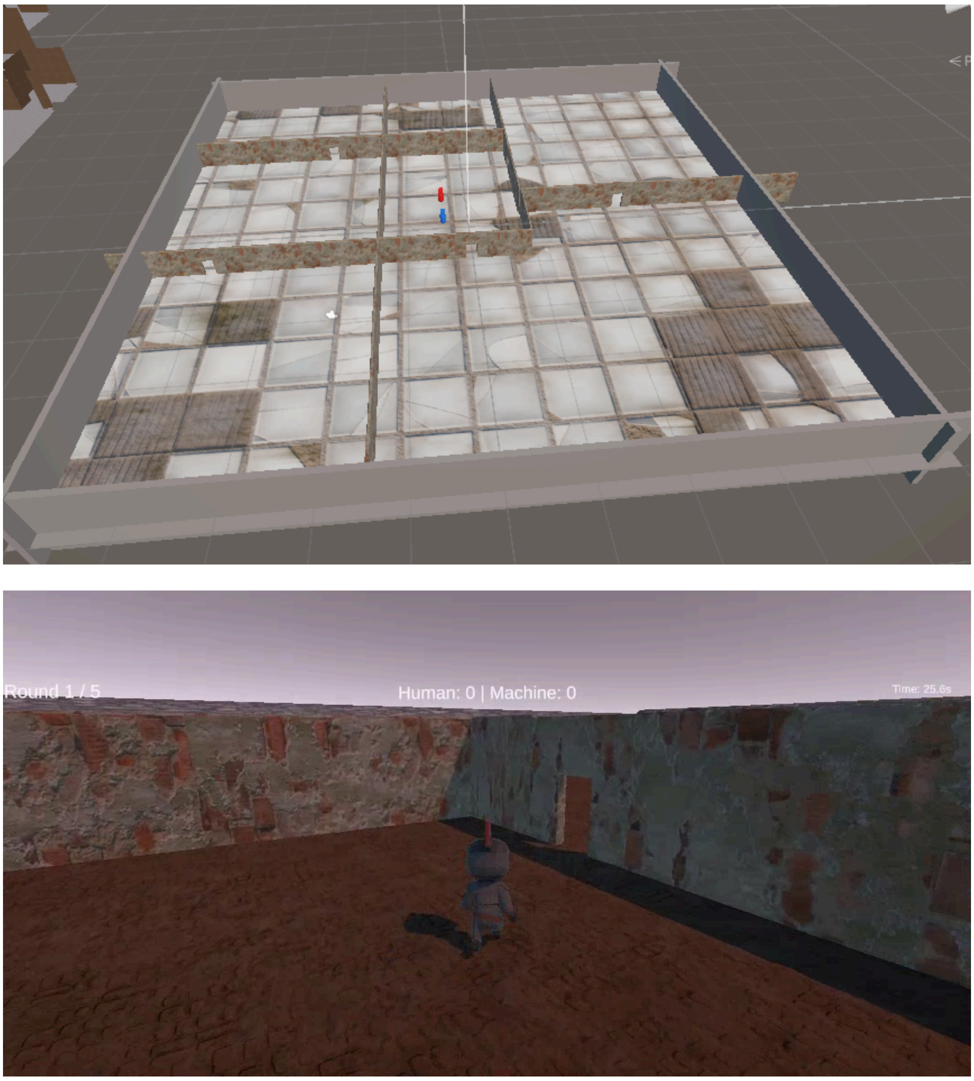

Figure 1: Arena 1: Top-down overview and ground-level capsule perspective.

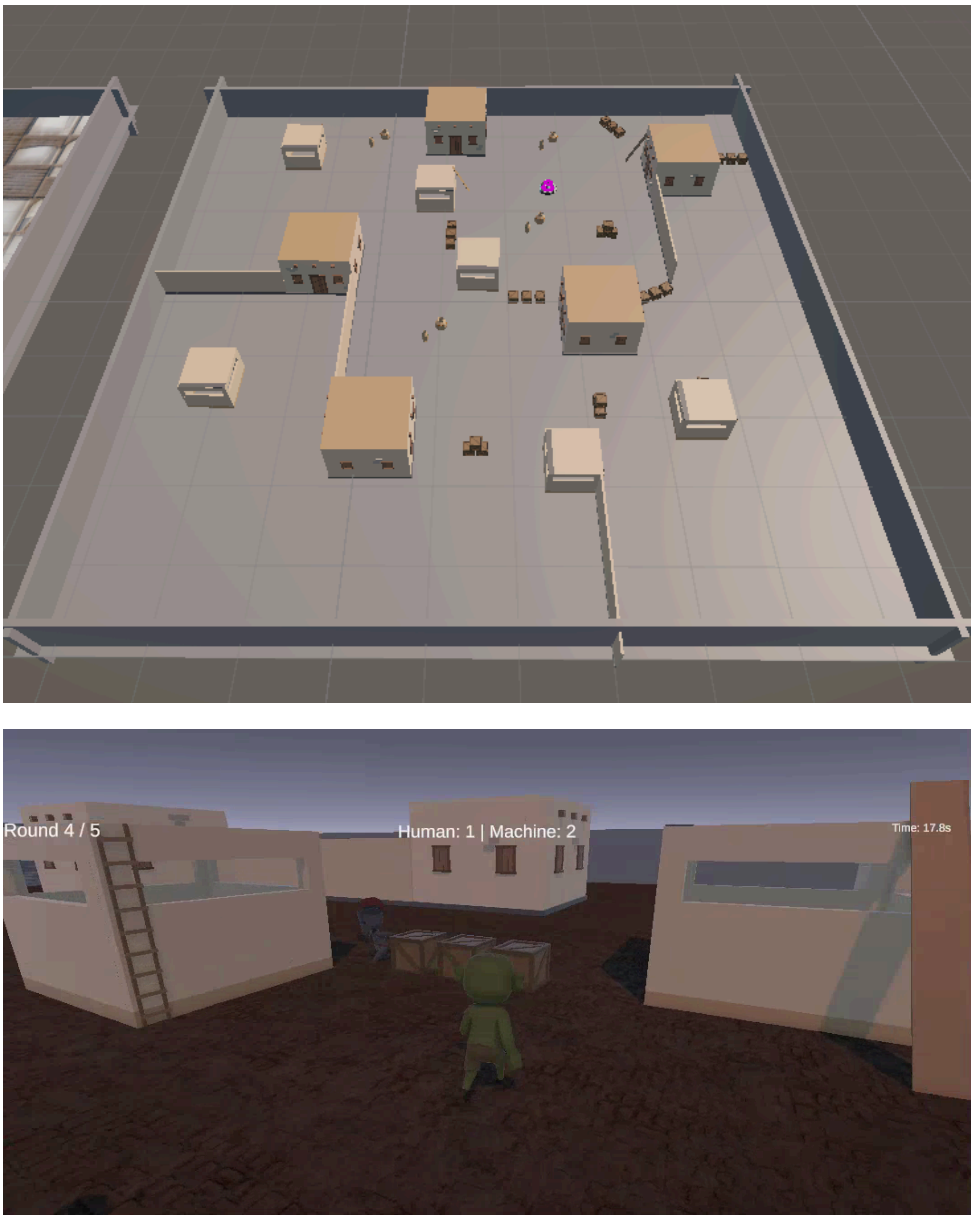

Figure 2: Arena 2: Top-down overview and interior structural view.

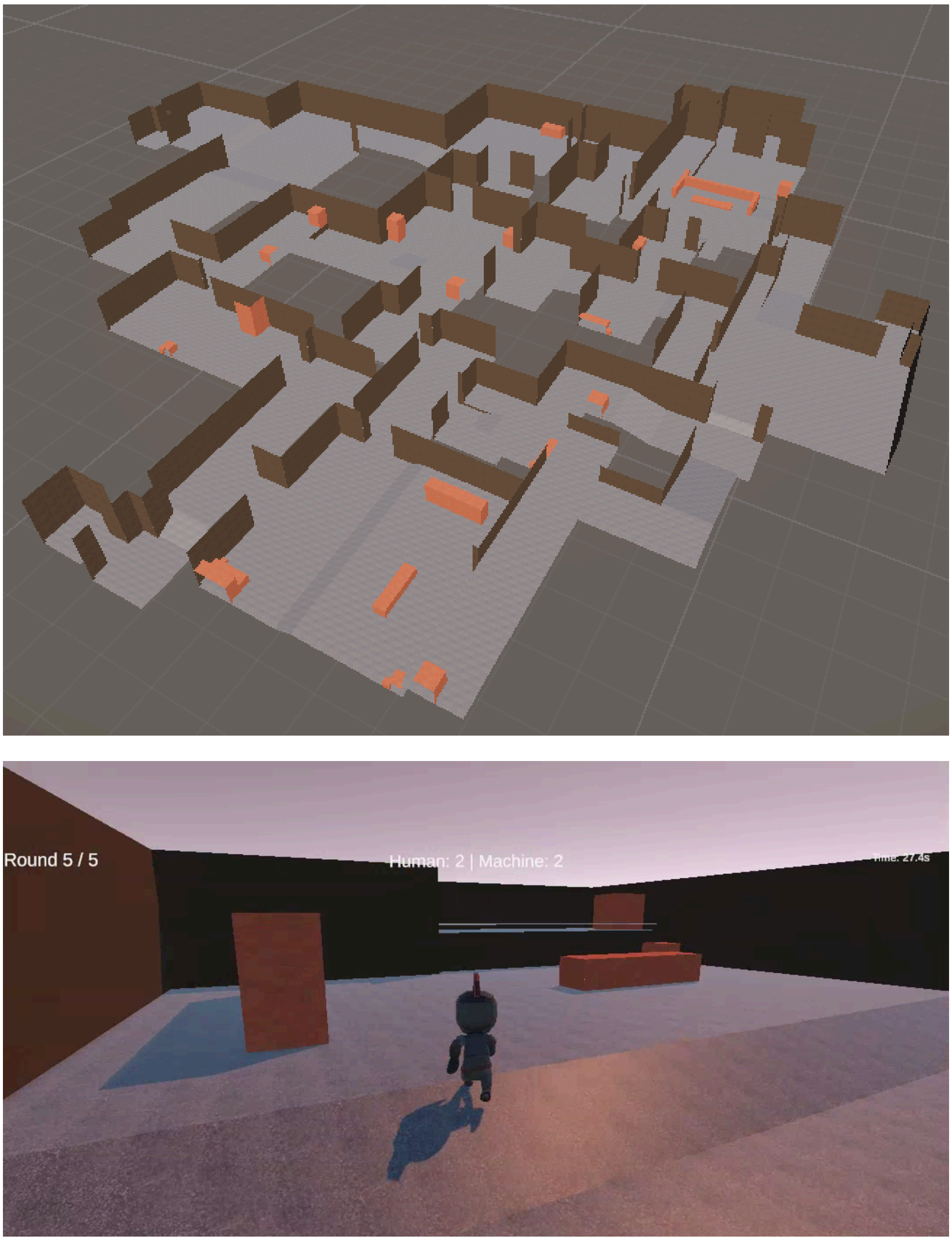

Figure 3: Arena 3: Final tiebreaker level overview and capsule perspective.

User Interface

It provides a streamlined graphical user interface managed directly by the `GameManager`. The UI is divided into several distinct states:

- **Start Screen:** The entry point of the game, allowing the player to initiate the match or access the rules.
- **Instructions Screen:** A dedicated panel explicitly detailing the objectives, the Best-of-5 round structure, and the keyboard controls.
- **Heads-Up Display (HUD):** Active during gameplay, it relays critical, real-time information, including the current round number, the remaining time in seconds, and the ongoing score between the Human and the Machine.
- **End Screen:** Upon match completion, it displays the ultimate victor and comprehensive session statistics, calculating the total rounds played and the average time spent per round.

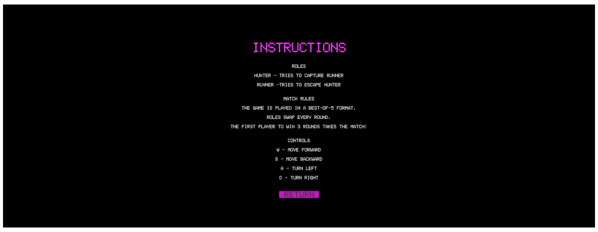

Figure 4: The main menu and the operational instructions interface.

Figure 5: The End Screen displaying the match victor and computed statistics.

Conclusion

"Alive by Nightfall" successfully demonstrates a functioning 3D asymmetric loop. By combining robust state management, responsive controllers, intelligent NavMesh-based opponent routing, and dynamic arena transitions, it provides a stable and replayable gameplay experience. The modular script architecture and isolated physics colliders lay a strong foundation for future iterations, including the potential expansion of the current heuristic-based AI into predictive machine learning models.

Credits

This project was made possible in part by the following assets and creators:

Mediterranean Ruins Building Kit by EmberStorm (3D models and textures)

Low Poly FPS Map Lite by JustCreate (3D models and textures)

Prototype Map (Map 3) by AngeloMaN87 (3D models)

Low Poly Park Pack by Fast Mesh (3D models)

World Materials by Avionx (Textures)

Character models by Quaternius